

BotSaaS

Documento de Arquitetura & Design

Plataforma SaaS de bot de IA para WhatsApp
C# .NET 10 · ASP.NET Core · MySQL · ADO.NET puro

Atualizado em 14/06/2026 · Fase 1 (Auth) concluída

Índice

- 1. Visão do produto
- 2. Stack tecnológica
- 3. Arquitetura em camadas
- 4. Estrutura de pastas
- 5. Módulos atuais
- 6. Modelo de dados
- 7. Decisões de arquitetura
- 8. Fluxo: Registro (transacional)
- 9. Fluxo: Login (JWT)
- 10. Segurança
- 11. Injeção de dependência (DI)
- 12. O produto em funcionamento (Fase 2)
- 13. Roadmap & estado atual
- 14. Próximos passos & pendências
- 15. Glossário

1. Visão do produto

SaaS de bot de IA para WhatsApp voltado a **pequenos negócios**. O cliente paga mensalidade e recebe um bot já configurado para o negócio dele, sem precisar saber de tecnologia.

Importante: o produto é uma **plataforma**, não "um bot". O bot é uma das funcionalidades dentro da plataforma. Cada loja é um *tenant* (uma *Company*), com um número, um bot e uma configuração própria.

Ator	Como usa	O que faz
Dono da loja	Painel web (futuro)	Cadastra-se, configura o bot, vê conversas, paga a mensalidade
Cliente final	WhatsApp	Manda mensagem para o número da loja e recebe resposta automática da IA
Você (SaaS)	Infra	Hospeda o banco e a aplicação; responsável pelos dados (LGPD)

2. Stack tecnológica

Camada	Escolha	Por quê
Backend	C# .NET 10, ASP.NET Core Web API (controllers)	Tipado, performático, ecossistema maduro
Banco	MySQL	Relacional, conhecido, barato de hospedar
Acesso a dados	ADO.NET puro + driver MySqlConnection	Sem ORM/Dapper — SQL escrito à mão, controle total, aprender o que está embaixo
Padrão de retorno	Result Pattern (mínimo)	Erro como valor, não como exceção espalhada
Auth	JWT próprio + BCrypt	Poucos usuários (donos), não precisa Auth0/Identity
Frontend	React + TypeScript (futuro)	—
WhatsApp	Evolution API ou Z-API (futuro)	Transporte abstraído, troca barata
IA	Anthropic Claude API (futuro)	O cérebro do bot
Config	.env (DotNetEnv)	Segredos fora do código

3. Arquitetura em camadas

Monolito **modular** (não microserviços). Três camadas com dependência de **mão única**:



Regra de ouro: Core depende de Shared, nunca o contrário. "Usado por todos" não significa "núcleo" — Shared é o chão, Core é o centro.

Como saber onde pôr cada coisa

- Fala de empresa / conversa / plano → **Core** (kernel).
- Serve qualquer app C# do mundo (Result, hash, conexão) → **Shared**.

- É específico de um nicho (barbearia, restaurante) → **Modules**.

4. Estrutura de pastas

```
src/
├── Core/                                ← KERNEL
│   ├── Auth/                          (autenticação: registro/login – orquestra o cadastro)
│   │   ├── AuthService.cs             IAuthService.cs
│   │   ├── AuthController.cs         AuthDTOs.cs (RegisterRequest, LoginRequest, UserResponse)
│   │   └── Users/                    (entidade User + gestão: perfil, listar)
│   │       ├── UserEntities.cs (User)
│   │       ├── UserService.cs        IUserService.cs
│   │       └── UserRepository.cs     IUserRepository.cs
│   ├── Companies/                     (os tenants)
│   │   ├── CompaniesEntities.cs (Company, enum Segment)
│   │   ├── CompaniesService.cs      ICompaniesService.cs
│   │   └── CompaniesRepository.cs   ICompaniesRepository.cs
│   ├── WhatsApp/ AI/ Conversations/ Billing/ ← Fase 2+ (vazias)
├── Modules/                          ← nichos que estendem o Core
│   └── Barber/                        ← Fase 3
└── Shared/                           ← base técnica
    ├── Database/ (IDatabase, MySQLDatabase, DbSession, DbCommandExtensions)
    ├── Security/ (IPasswordHasher, BcryptPasswordHasher, IJwtTokenGenerator, JwtTokenGenerator)
    └── Results/ (Result<T>)
```

Obs.: namespace é lógico (`BotSaaS.Api.Core.Users`), ignorando o `src/`. A pasta é organização; o **namespace** é o que define o módulo de verdade.

5. Módulos atuais

Módulo	Responsabilidade	Conteúdo
Auth	Autenticação — registro e login. Orquestra o cadastro (cria Company + User numa transação).	AuthService, AuthController, DTOs
Users	Dono da entidade User e da gestão de usuário (criar, buscar por email; futuro: perfil, listar).	User, UserService, UserRepository
Companies	Os tenants. Dono de "o que é uma Company válida". Futuro: BotConfig, settings.	Company, CompaniesService, CompaniesRepository

Por que Auth e Users são separados? Autenticação (login/registro/senha) é um concern; a entidade User e sua gestão (perfil, listar) é outro. `Auth` **depende de** `Users`, mão única.

6. Modelo de dados

Relações

```
Company 1—N User      ✓ existe (tenant + donos)
Company 1—N Channel   ⌚ Fase 2 (WhatsApp/Telegram...)
Channel 1—N Conversation ⌚ Fase 2
Conversation 1—N Message ⌚ Fase 2 (Role: User / Assistant)
```

Tabelas (criadas)

companies		users	
id	CHAR(36) PK	id	CHAR(36) PK
name	VARCHAR(150)	company_id	CHAR(36) FK → companies(id)
segment	INT	name	VARCHAR(150)
created_at	DATETIME	email	VARCHAR(255) UNIQUE (global)
		password_hash	VARCHAR(255)
		created_at	DATETIME
todas NOT NULL		todas NOT NULL	

email UNIQUE global: um e-mail = uma conta. É essa constraint que gera o erro 1062 (duplicate) no cadastro repetido — e que o teste de atomicidade usou para provar o rollback.

Mapeamento C# → MySQL

C#	MySQL	Detalhe
Guid	CHAR(36)	Texto legível; evita conversão de bytes do BINARY(16) no ADO.NET puro
enum	INT	Valor numérico; (int)e ↔ (Enum)reader.GetInt32(). Nunca reordenar os números.
DateTime	DATETIME	Preenchido pela aplicação (DateTime.UtcNow), não pelo banco
Coluna: snake_case sem prefixo (created_at, não comp_created_at). SELECT sempre com colunas explícitas.		

7. Decisões de arquitetura

7.1 Repositório burro (Escola A)

O repository tem **uma** responsabilidade: falar SQL. Ele **roda SQL ou estoura** a exceção do driver. Sem try/catch, sem Result lá dentro. Retorna dado cru, nunca Result.

- Insert/Update/Delete → Task
- Buscar um → Task<User?> (null = não achou, caso normal)
- Buscar vários → Task<List<T>> (lista vazia, nunca null)

7.2 Camadas de erro (contrato)

```
Repository → estoura a exceção do driver (sobe como DbException)
|
Service → captura DbException (base abstrata, NÃO a MySqlConnection concreta),
|         valida regra, devolve Result com código semântico
|         [não conhece HTTP nem idioma]
|
▼
Controller → traduz o código em status HTTP (400/401/404/409) + mensagem
|         [único que conhece HTTP]
```

Capturar DbException (de System.Data.Common) e não MySqlConnection mantém o **Core desacoplado do driver** — mesmo princípio do IDatabase.

7.3 Atomicidade do registro = transação via DbSession

Registrar cria **Company + User**: ou nascem os dois, ou nenhum. Como uma transação vive numa **única conexão**, os dois INSERTs precisam compartilhá-la.

- `DbSession` (Scoped) — caixa que segura `Connection` + `Transaction` da requisição.
- `AuthService` é o **dono do ciclo**: cria a conexão, abre, `BeginTransaction`, enche o `DbSession`, e no fim `Commit` (ou `Rollback` no catch).
- Os repos injetam o **mesmo** `DbSession` (Scoped = mesma instância) e leem dele → os 2 INSERTs caem na mesma transação sem passar nada por parâmetro.

Descartado por ora: passar a transação por parâmetro (mais explícito, repete) e Unit of Work com comportamento. Quando vários services dividirem transação, o `DbSession` vira UoW.

7.4 Interface neutra (esconder a tecnologia)

Toda tecnologia específica fica atrás de uma **interface com nome neutro**; a marca aparece só na implementação:

Interface (neutra)	Implementação (marca)	Esconde
<code>IDatabase</code>	<code>MySQLDatabase</code>	<code>MySQLConnector</code>
<code>IPasswordHasher</code>	<code>BcryptPasswordHasher</code>	<code>BCrypt</code>
<code>IJwtTokenGenerator</code>	<code>JwtTokenGenerator</code>	<code>System.IdentityModel.Tokens.Jwt</code>

Trocar a tecnologia (MySQL→Postgres, BCrypt→Argon2) custa **uma linha na DI**.

7.5 Régua de nome por camada

Camada	Verbo	Exemplo
Repository	de dado (espelha SQL)	<code>InsertUser</code> , <code>GetUserByEmail</code>
Service	de domínio	<code>CreateUser</code> , <code>CreateCompany</code>
AuthService	caso de uso	<code>Register</code> , <code>Login</code>

O repo **nunca** leva nome de caso de uso (nada de `RegisterUser`).

8. Fluxo: Registro (transacional)

```
POST /api/auth/register { ownerName, email, password, companyName, segment }
|
v
AuthController ..... valida HTTP, chama o service
|
v
AuthService.Register ..... DONO da transação
| 1) cria conexão (IDatabase) → enche DbSession → OpenAsync → BeginTransaction
|
| → CompaniesService.CreateCompany(name, segment)
|   ↳ CompaniesRepository.InsertCompany → INSERT companies
|
| → UserService.CreateUser(name, email, password, companyId)
|   ↳ hash da senha (IPasswordHasher.Hash → BCrypt)
|   ↳ UserRepository.InsertUser → INSERT users
|
| 2) Commit (deu tudo certo)
|   catch DbException → Rollback (desfaz tudo, sem Company órfã)
|
v
Result<User> → Controller mapeia para UserResponse (SEM o hash) → 200 OK
```

Provado em teste e2e: registro → 200; email repetido → 400 + rollback confirmado no banco (nenhuma Company órfã). A atomicidade funciona.

9. Fluxo: Login (JWT)

```
POST /api/auth/login { email, password }
|
v
AuthController
|
v
AuthService.Login ..... leitura, SEM transação
| ↳ abre conexão (enche DbSession.Connection, sem BeginTransaction)
| ↳ UserService.GetUserByEmail(email) → UserRepository → SELECT users
|   ↳ null? → Result.Failure("Credenciais inválidas")
| ↳ IPasswordHasher.Verify(senha, hash)
|   ↳ inválida? → Result.Failure("Credenciais inválidas") ← MESMA msg (não revela qual)
| ↳ IJwtTokenGenerator.GenerateToken(userId, companyId, email)
|
v
Result<string> (o token) → 200 { token } | falha → 401 Unauthorized
```

10. Segurança

Item	Como
Senha	Nunca em texto puro. BCrypt (work factor 11, salt embutido) → <code>\$2a\$11\$...</code> . Irreversível.
Login	<code>Verify</code> compara a senha com o hash. Mensagem genérica para "email inexistente" e "senha errada" (não revela qual).
Token	JWT assinado com HMAC-SHA256 e a <code>JWT_SECRET</code> . Expira em 8h.
Claims	<code>sub</code> (userId), <code>companyId</code> , <code>email</code> . Nunca senha. São assinados, mas legíveis (base64) — nada sensível ali.
Multi-tenant	O <code>companyId</code> vem no token; toda query futura filtra por ele (isolamento entre lojas).
Segredos	<code>.env</code> (DB e <code>JWT_SECRET</code>). Vai para o <code>.gitignore</code> quando iniciar o git.

Atenção: claims são **assinadas, não criptografadas**. Qualquer um decodifica e lê o conteúdo. A assinatura só garante que ninguém alterou. Por isso: nada de segredo nas claims.

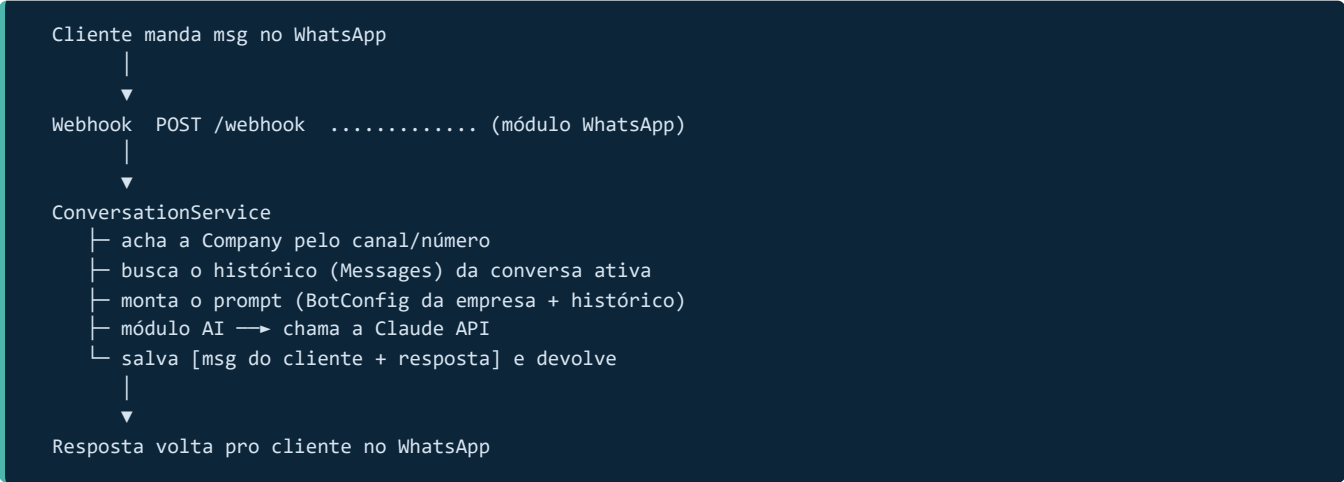
11. Injeção de dependência (DI)

Lifetime	Significado	Quem
Singleton	1 instância para o app inteiro (sem estado)	<code>IDatabase</code> , <code>IPasswordHasher</code> , <code>IJwtTokenGenerator</code>
Scoped	1 instância por requisição	<code>DbSession</code> , todos os repos e services, <code>AuthService</code>
Transient	1 nova a cada pedido	(não usado ainda)

O `DbSession` ser **Scoped** é o que faz o truque da transação funcionar: numa requisição, o `AuthService` e os repos recebem a **mesma** caixa, então o que um enche o outro lê.

12. O produto em funcionamento (Fase 2)

Hoje existe só a **porta de entrada** (auth). O coração do produto vem na Fase 2:



- ## 13. Roadmap & estado atual

Feito na Fase 1

14. Próximos passos & pendências

- ## TODOs no código

- `AuthService` ainda devolve `Result.Failure("Erro ao registrar")` (string) → virar código semântico.
- `.env` precisa entrar no `.gitignore` quando o git for iniciado.

15. Glossário

Termo	O que é
Multi-tenant	Vários clientes (lojas) no mesmo banco, isolados por <code>CompanyId</code> .
Transação (ACID)	Conjunto de operações "tudo ou nada": <code>BEGIN</code> → <code>COMMIT</code> (grava) ou <code>ROLLBACK</code> (desfaz).
DI (Injeção de Dependência)	Receber colaboradores prontos em vez de criá-los; o container monta tudo.
Repository	Camada que só fala SQL com o banco. "Burra" — sem regra de negócio.
Service	Camada de domínio: regras, orquestração. Não conhece HTTP.
DTO	Objeto de transporte na borda HTTP (request/response). Ex: <code>RegisterRequest</code> , <code>UserResponse</code> .
<code>Result<T></code>	Embrulho de retorno: sucesso (com valor) ou falha (com erro). Genérico — <code>T</code> é o tipo do valor.
Hash (BCrypt)	Transformação irreversível da senha, com salt. Não dá pra "descriptografar".
JWT	Token assinado que prova quem é o usuário sem consultar o banco a cada request.
Claim	Uma afirmação dentro do token (id, companyId, email).
Genérico (<T>)	Tipo "preenchido no uso": <code>Result<User></code> , <code>Result<string></code> .